

Neural Models and Text Classification

Core Pipeline: Data Acquisition -> Text Cleaning -> Pre-Processing -> Feature Extraction -> Machine Learning -> Evaluation and Deployment

Techniques: Tokenization, Lowercasing, Stopword removal, Stemming or Lemmatization. Golden Rule: Standardization helps generalization. **Trade-off:** You lose nuanced information.

Naive Bayes (Generative): Models the joint probability $p(x, y)$ by calculating the prior $p(y)$ and likelihood $p(x \text{ given } y)$. Words are conditionally independent given the label y . **Formulas:** Prior: $p_i = \text{count}(y = i) / n$ Likelihood: $\text{tau_wj} = \text{count}(w \text{ in } j) / \text{total_words in } j$ Laplace Smoothing: Prevents zero probabilities for unseen words. Formula: $\text{tau_wj} = (\text{count}(w, j) + \text{lambda}) / (\text{count}(j) + |V| * \text{lambda})$. **Pros:** Easy to learn (just counting), works well with small datasets.

Numerical Underflow: Work in log space to avoid multiplying many small numbers into zero.

Logistic Regression (Discriminative): Directly models $p(y \text{ given } x)$ by optimizing parameters (w, b) to discriminate between classes. Linear Model: Score $s = w^T \text{phi}(x) + b$ **Sigmoid Function:** $\text{sigma}(z) = 1 / (1 + e^{-(z)})$ Maps scores to probabilities between 0 and 1. Objective: Minimize the loss function $L(w, b) = \text{sum from } i = 1 \text{ to } n \text{ of } -\log \text{sigma}(y^{(i)} * (w^T \text{phi}(x^{(i)} + b)))$. **Pros:** Usually yields higher accuracy on large datasets, easier to add arbitrary features.

Convexity: The loss function is bowl-shaped (convex), so gradient descent finds the global minimum.

Unigrams (Bag of Words): Counts single words. Ignores word order. Simple and efficient. Loses sentence structure and context. **Bigrams:** Uses two-word sequences. Captures short context such as "not good". Parameter count increases dramatically (up to vocabulary size squared). Risk of overfitting.

Evaluation Metrics: Data Splits: Training Set (train model), Development Set (tune hyperparameters), Test Set (final evaluation). **Accuracy:** Correct divided by Total (poor for imbalanced labels) (TP+TN/Total). **Precision:** True Positives divided by Predicted Positives (TP/TP+FP) **Recall:** True Positives divided by Actual Positives (TP/TP+FN) **F1 Score:** Harmonic mean, excellent for label imbalance $(2 * P * R / (P + R))$

Word Vectors (Word2Vec)

Distributional Hypothesis: Words appearing in similar contexts tend to have similar meanings. Goal: Map each word in the vocabulary to a dense, low-dimensional vector that captures lexical semantics such as synonymy (car/automobile), antonymy (hot/cold), hypernymy/hyponymy (animal/dog), and similarity.

Creating Word Vectors: Co-occurrence Counting - Using a sliding window, count neighboring words to build a term-term matrix. Problem: This produces very high-dimensional, sparse vectors.

Word2Vec (Dense Vector Learning) - Instead of directly using counts, Word2Vec learns dense embeddings through a "fake" supervised learning task.

Training Process: Use a sliding window of size k . For each center word w , create positive pairs $(w, \text{context word})$. Generate negative examples by randomly sampling words from the vocabulary. Negative sampling uses alpha-weighted frequency (typically $\alpha = 0.75$) to reduce over-sampling of frequent words like "the". **Maximize** the dot product for real (positive) pairs. **Minimize** the dot product for fake (negative) pairs. Uses two sets of parameters: Word vectors v_w (feature vectors). Context vectors c_w (helper can be dropped later).

Optimization: Similar to logistic regression. In logistic regression, input features are fixed and only weights are learned. In Word2Vec, both word vectors and context vectors are learned. The objective is **non-convex because it optimizes both simultaneously**. Gradient descent still works well in practice.

Cosine Similarity: Used to measure similarity between word vectors. $\text{cos}(\theta) = (u \cdot v) / (||u|| ||v||)$ 1: same direction (very similar); 0: orthogonal (unrelated); -1: opposite direction. Cosine similarity measures direction, not magnitude.

Word vectors capture relational structure. Example: $v_{\text{king}} - v_{\text{man}} + v_{\text{woman}} \approx v_{\text{queen}}$

To find X where A is to B as C is to X : $v_X \approx v_B - v_A + v_C$

Mean Pooling: Average all word vectors in a document to create a single document representation. Limitation: This treats text as bag-of-words and ignores word order and structure.

Strengths of Word Vectors Generalization: Helps models understand rare or unseen words. Word2Vec is often pretrained on large corpora such as Wikipedia, news articles, and books before training a task-specific classifier.

Limitations: 1. **Sparsity** (for count-based methods): Co-occurrence matrices are high-dimensional and sparse. 2. **Bias:** Word2Vec captures statistical patterns from data, including historical and social biases (e.g., gender-occupation stereotypes). 3. **Polysemy:** Standard Word2Vec assigns one vector per word. Cannot distinguish different meanings of the same word (e.g., "bank" as river bank vs. financial bank). 4. **Word Order:** Averaging word vectors ignores sentence structure and word interactions.

Neural Networks

Neural networks are learnable non-linear functions. Without non-linear activation functions between layers, a multi-layer network collapses into a single linear function: $v^T(Wx + b) + c = (v^T W)x + (v^T b + c)$

Activation Functions: **ReLU:** $\text{ReLU}(x) = \max(x, 0)$. Very fast to compute. **Tanh:** Preferred over sigmoid because outputs are centered around zero. **Sigmoid:** $\text{sigma}(z) = 1 / (1 + e^{-(z)})$.

Non-linearity is essential to learn complex, non-linearly separable functions. Linear models cannot solve problems that are not linearly separable, but neural networks can.

Layer Roles: **Hidden layer:** Acts as a feature extractor. Each neuron behaves like a logistic regression classifier with parameters w_j and b_j for $j = 1$ to h (h = number of hidden units). Hidden layers produce learned feature vectors (activations). **Final layer:** Binary classification: Logistic regression-like layer with weight vector v and bias c . Multi-class classification: Linear model with weight vector V_k and bias c_k for each class k , usually followed by softmax. Only the final layer changes when adapting the model to different tasks.

Matrix Form: W : $h \times d$ matrix; b : length h vector. Model parameters: $\theta = (W, b, v, c)$

Deep networks (more layers, more layers) often perform better than simply making networks wider. Neural networks learn new features from data. Each learned feature is the output of a linear classifier over input features. These features help the final layer make better predictions.

Deep learning focuses on designing suitable neural networks using reusable building blocks. **Core Building Blocks:** **Linear block:** Input: x (dimension d_{in})

Output: y (dimension d_{out}) Formula: $y = Wx + b$ Parameters: W ($d_{out} \times d_{in}$), b (d_{out} vector) In PyTorch: `nn.Linear()`

Non-linearity block: Input: vector/matrix. Output: same shape Parameters: none

Word vector block: Input: word w Output: vector of length d Parameters: one vector per word; total parameters = $|V| \times d$

Mean pooling block: Input: matrix of shape (d, L) Output: vector of dimension d Operation: average across the L dimension Parameters: none

Epoch: One epoch is one complete pass over the entire training dataset. In SGD, data is divided into batches, but an epoch completes only after all batches are processed.

Batch Size: Larger batch size: More accurate gradient, slower computation. Smaller batch size: Faster updates, noisier gradients.

Convex vs Non-Convex: Linear models are convex: All local optima are global optima. Initialization does not significantly affect the final result. Parameters can be initialized to zero. Neural networks are non-convex: Hard to find global optima. Different initializations and optimization methods can lead to different solutions.

Symmetry Problem: If all weights are initialized to zero, all hidden units behave identically because their gradients are identical. They will remain the same. To break symmetry, initialize with small random values.

Learning Rate: Too small: Slow convergence, may get stuck in flat regions. Too large: Erratic/dramatic updates may not converge. Common strategy: Start large and decay over time.

Challenges with SGD: Ill-conditioned loss surfaces: Very steep in some directions and flat in others. Large learning rate: Zig-zag in steep directions. Small learning rate: Very slow progress. **Saddle points:** Regions where gradient is near zero but not a minimum; training slows down. Improvements: **Momentum:** Keeps a running average of past gradients. Reduces zig-zagging. Builds speed in consistent directions. Helps move through flat regions.

Per-parameter learning rates: Different step sizes for each parameter. Small steps in steep directions, larger steps in flat ones. Examples: Adagrad, RMSProp.

Adam: Combines Momentum and RMSProp. Most commonly used optimizer.

Deep Averaging Network (DAN): Maps words to vectors. Average them (mean pooling). Pass through an MLP (Multi Layer Perceptron). Often outperforms Naive Bayes on sentiment analysis.

CNN for Text: Train a neural network to look at fixed-size windows (e.g., 3 words). Apply the same network across all windows. Mean pool the resulting features. Creates a neural "bag of n-grams". Advantages over traditional n-grams: Word vectors allow processing unseen n-grams because similar words have similar vector representations. Exact word sequences are rare, so vector-based generalization is powerful.

Backpropagation: Backpropagation is an algorithm for computing gradients of the loss with respect to all parameters. It applies the chain rule to differentiate through each layer (block). It enables efficient training of deep networks.

Sequence Labeling

Structured Prediction: Predicting an output $Y = \{y_1, y_2, \dots, y_n\}$ where components are strongly correlated. The output space is **exponential** because multiple components interact. **Unstructured Prediction:** Output Y consists of a single component.

Sequence Labeling: Assigning a label y_i to each token x_i in a sequence. Examples: POS tagging, Named Entity Recognition (NER), translation, auto-completion. The sequence of words plays a crucial role.

Modeling Goal: Instead of predicting each tag independently, we model $P(Y | X)$ directly to capture dependencies between tags. **Most Frequent Tag baseline** achieves ~92% token accuracy. **Sentence accuracy is much lower** due to error compounding (e.g., $0.9214 \times 31\% \times 60.92\% \times 14\% \approx 31\% \times 60.9214 \times 31\%$). **Highly domain dependent** — works best when train and test data match. **Ignores context**, leading to feature sparsity issues. **Tag dependencies matter** (e.g., adjective → determiner is unlikely). Accurate POS tagging requires **global sentence-level modeling**, not independent word predictions.

POS Tag Categories: **Closed class:** Fixed membership, function words (prepositions, determiners). **Open class:** Frequently updated (nouns, verbs, adjectives, adverbs).

Types: Unique words, Tokens: Actual occurrences of words.

Sequence Encoder: CNNs: Pros: Simple architecture, parallel computation. Cons: Limited context window, weak at modeling long-range dependencies. Can process fixed-size windows and create neural "bag of n-grams". **Simple RNNs:** Reuse a feedforward network recurrently. Formula: $t = \tanh(W_x x + W_h h_{t-1} + b)$ Problems with RNNs: No access to future context. Sequential computation (inefficient). Gradient vanishing/exploding due to repeated multiplication of weights. Limited hidden state size (memory constraints). **Bidirectional RNNs:** Process sequence in both forward and backward directions. Solve the "no future context" problem. Applicable when the full input sequence is available. Common choice: Multi-layer bidirectional RNNs. Essential for sequence labeling tasks.

LSTMs (Long Short-Term Memory): Designed to solve gradient vanishing/exploding. Use gating mechanisms: Input gate; Forget gate; Output gate. Maintain a memory cell to preserve long-range information. **Bi-LSTM-CNN:** Combines: Character-level CNN for spelling/prefix features. Word-level Bi-LSTM for contextual representation. Produces robust word representations.

Log-Linear and Structured Models: Log-Linear Models: Define feature functions $f(x, y)$ that measure compatibility between input x and label y . Compute: Score = $v \cdot f(x, y)$ Exponentiate and normalize to get probabilities. The score is linear in weights, but normalization uses exponentials and logs, hence the name log-linear. Feature Explosion: Combining many features (word + tag, word + context) leads to a huge, sparse feature space. Most features occur rarely, making training difficult.

Independence Assumptions: To make computation tractable, we introduce independence assumptions. Common assumption: Each tag depends only on the previous tag (Markov assumption).

MEMMs vs CRFs: Even with strong neural features, we must model dependencies between output tags.

MEMMs (Maximum Entropy Markov Models): Locally normalized. Predict tag at position j given previous tag and full input. Normalize at each position separately. Use Markov assumption. **Problem:** Label Bias If a tag has very few outgoing transitions (e.g., DT almost always followed by NN), MEMM assigns very high probability locally, even if globally incorrect. Because normalization is local, states with fewer outgoing transitions are favored.

CRFs (Conditional Random Fields): Globally normalized. Score the entire sequence first. Normalize over all possible full sequences. Weaker independence assumptions than MEMMs. Avoid label bias by considering the full path before

normalization. CRF models $P(s_1, \dots, s_m | x_1, \dots, x_m)$ directly. Structured models explicitly capture dependencies between output tags.

Modern NLP: With powerful contextual embeddings, sometimes independent tag prediction works well. However, CRFs are still useful when strong label dependencies exist.

State-of-the-Art Classical Model: BiLSTM-CNN-CRF CNN: Character-level features. BiLSTM: Contextual word representations. CRF: Structured output dependencies.

Decoding: The Viterbi Algorithm: After defining a CRF, we must find the most probable tag sequence. We do not enumerate all sequences (exponential). Instead, we use dynamic programming.

Viterbi Algorithm: At each position j ; Keep the best path ending in each possible tag. Recursively compute:

$\text{pi}[j, s] = \max \text{over } s' \text{ of } [\text{pi}[j-1, s'] \times \text{transition_score}]$ Move forward through the sequence. At the end, backtrack to recover the best sequence. This efficiently finds the highest-scoring tag sequence.

Seq2Seq and Neural Machine Translation (NMT)

Seq2Seq vs Sequence Labeling: Sequence labeling: Output length equals input length. One label per input token. Seq2Seq: Input length (L) and output length (T) can differ. Seq2Seq output space is much larger. No strict one-to-one word alignment. Output tokens depend on previously generated output tokens.

Statistical Machine Translation (SMT): Used word-alignment models. Words are likely aligned if they frequently co-occur in translation pairs.

Autoregressive Factorization: Uses the chain rule to model the joint probability: $p_{\theta}(Y | X) = \text{product over } t=1 \text{ to } T \text{ of } p_{\theta}(y_t | y_{<t}, X)$. Each word is predicted using all previous output words and the input. Does not assume independence. "Auto" means self, "regressive" means using previous outputs.

Encoder-Decoder Architecture: **Encoder:** Converts the input sequence into a sequence of vectors (hidden states). **Decoder:** Converts encoded vectors into an output sequence. Generates tokens one at a time. To generate token y_t , it uses: Input X (via encoder outputs). Previously generated tokens $y_{<t}$. **Training:** Maximum Likelihood Estimation (MLE): Maximize product of $P(y_t | y_{<t}, X)$. Teacher Forcing: During training, feed the true previous word instead of the model's prediction. Gradients flow from decoder back into encoder, so both learn jointly.

Special Tokens: <BOS> Begin of sequence. <EOS> End of sequence. Generation stops when it is produced or max length is reached.

Decoding Algorithms: After training, we must find the best output sequence.

Exhaustive Search: Try all possible sequences. If vocabulary size = V and output length = T : Number of sequences = V^T Computationally impossible. **Greedy Search:** At each step: $y_t^* = \text{argmax}_y p_{\theta}(y_t | y_{<t}, X)$. Picks most probable word at each step. Fast. Suboptimal: Best local choice may not give best global sequence. **Beam Search:** Keep top K most probable partial sequences (hypotheses) at each step. Score each hypothesis using log probabilities. Probabilities multiply. Logs convert multiplication into addition. More efficient than exhaustive search. Not guaranteed optimal. Can fail if correct path falls out of top K early. Beam search is the practical middle ground.

Problem with Vanilla Encoder-Decoder: Bottleneck Problem: Encoder compresses entire source sentence into a single vector. Difficult to capture long sentences. Causes information loss. Leads to vanishing gradient problems on long sentences.

Attention Mechanism: At each decoding step, instead of using a single fixed vector, look at all encoder states and decide which parts are important. Encoder outputs: $c_1, c_2, \dots$, on Decoder hidden state at time t : h_t

Step 1: Compute Attention Scores For each encoder state c_j : $e_{tj} = \text{sim}(c_j, h_t)$ This measures how relevant input position j is for generating y_t . **Step 2: Softmax** **Distribution** $\alpha_t = \text{softmax}(e_t)$ Converts scores into probabilities that sum to 1. α_{tj} represents how much attention is given to word j . **Step 3: Attention Output** Compute weighted sum: $\text{attention_output} = \text{sum over } j=1 \text{ to } n \text{ of } \alpha_{tj} * c_j$ This context vector is used by the decoder to generate the next word.

Attention solves: Bottleneck problem. Improves handling of long sentences. Reduces information loss.

Types of Attention Scoring Functions: Classification is based on how similarity $\text{sim}(c_j, h_t)$ is computed.

Dot-Product Attention: $\text{sim}(c_j, h_t) = c_j^T h_t$. Assumes equal dimensions. Simple and fast. **Multiplicative Attention:** $\text{sim}(c_j, h_t) = c_j^T W h_t$. W is a learnable weight matrix. More flexible than simple dot product. **Additive Attention:** $\text{sim}(c_j, h_t) = v^T \tanh(W_c c_j + W_h h_t)$. W_c and W_h are learnable matrices. v is a learnable vector. More expressive but slightly more computationally expensive.

Pytorch

A **Tensor** is a multi-dimensional array containing elements of a single data type. It is similar to a NumPy array but can run on CPU or GPU (CUDA). **Storage:** Points to contiguous or strided memory **Metadata:** Shape, dtype, stride, device

Device-agnostic: Works on CPU or CUDA **Creating Tensors:** From Python lists: `torch.tensor([...])`. From NumPy: `torch.from_numpy(np_array)`. From another tensor: `old_tensor.detach().clone()` or `old_tensor.clone()`

Matrix Multiplication: `@` operator; `torch.matmul()`; `torch.mm()` (2D only); `torch.bmm()` (3D only); `torch.einsum()`

Useful Tensor Methods: `item()` → converts single-value tensor to Python float; `clamp()` → limits values to a range; `t()` → transpose (2D)

Autograd automatically computes gradients using the chain rule. Without Autograd, you would need to manually compute gradients, impractical for deep learning. Enable gradient tracking: Set `requires_grad=True`; PyTorch builds a **Dynamic Computational Graph (Define-by-Run)** during the forward pass. Each operation attaches a `grad_fn` to the output tensor. Calling `loss.backward()` traverses the graph backward. The graph is destroyed after backward. Important Concepts:

Node: Tensor Edge: Function `grad_fn`: Backward pointer to compute gradients

Gradients accumulate by default

Gradient Reset: Use `model.zero_grad()` or `param.grad.zero_()`

Weight Updates: Wrap updates inside with `torch.no_grad()`: Prevents weight updates from being tracked in the graph Avoids incorrect gradient computation

Why torch.no_grad() is critical: If weight updates are tracked, PyTorch may attempt to compute gradients through the update step itself, leading to errors or incorrect gradients.

Custom Autograd: Use `torch.autograd.Function` when: The operation is non-differentiable by default. You need a custom backward rule. You want

